

Criteria for All Individual React Projects

"No matter which project you build, you MUST include these features."

1. Component Architecture

Every project must:

- Break UI into at least 3 reusable components
- Use:
 - Parent component
 - Child components
- Follow:
 - `App` -> Container -> UI Components`

Example:

- App
 - TaskList
 - TaskItem
-

2. State Management (useState Required)

Every project must:

- Use `useState` for dynamic data
 - Manage:
 - Input state
 - Display state
 - Avoid global variables
-

3. Event Handling

Must include at least:

- `onClick`
- `onChange`
- One custom handler function

Example:

- Add button
 - Delete button
 - Input typing
-

4. Props Usage

Every project must:

- Pass data using props
- Pass at least one function via props

This ensures:

- Parent -> Child communication
 - Real React understanding
-

5. List Rendering + Keys

If project involves data:

- Use `.map()` to render lists
 - Use unique keys
-

6. LocalStorage (MANDATORY)

Every project must:

Store data:

```
EXAMPLE
localStorage.setItem("key", JSON.stringify(data))
```

Retrieve data:

```
EXAMPLE
JSON.parse(localStorage.getItem("key"))
```

Must use with:

- `useEffect`

Example:

- Save todos
- Save theme
- Save form inputs

- Save game scores
-

7. useEffect (Side Effects)

Each project must:

- Use `useEffect` at least once

Common uses:

- Load data from `localStorage`
 - Fetch API
 - Timers
-

8. API Handling (If Applicable)

For API-based apps:

Must include:

- `fetch` or `axios`
 - Loading state
 - Error handling
-

9. Clean UI + Conditional Rendering

Must include:

- `if` / ternary (`? :`)
- Show/hide elements

Example:

- Show "No Data"
 - Show Loader
 - Toggle UI
-

10. Basic Validation

Every project must validate:

- Empty inputs
 - Invalid input (if applicable)
-

11. CRUD Operations (if data-based)

Projects like:

- To-do
- Notes
- Contacts

Must include:

- Create
 - Read
 - Update
 - Delete
-

12. Styling (Minimum Standard)

Must include:

- Basic CSS OR Tailwind
 - Clean spacing
 - Button styling
-

13. File Structure Discipline

Students must organize:

```
/src
  /components
  /pages (if needed)
  /utils (optional)
```

14. Reusability Rule

At least one reusable component

Example:

- Button component
 - Card component
-

15. Bonus

- Custom hook

- Context API
- Debouncing (search apps)

Evaluation Rubric

Criteria	Marks
Functionality	30
React Concepts	25
Code Structure	15
LocalStorage Use	10
UI/UX	10
Bonus Features	10

“Don't just build the app - build it like a **real product**.”

BEST OF LUCK